

Sistemi Operativi (Modulo II)

Capitoli 3 - 4 — Scheduling e risorse

Luca Argentino

A.A. 2025–2026

Indice

3	Scheduling, Processi e Thread	2
3.1	Il Sistema Operativo come Gestore di Risorse	2
3.1.1	Il Descrittore di Processo (PCB)	2
3.2	Ciclo di Vita di un Processo	3
3.2.1	Vita di un Processo nello Scheduler: Gestione delle Code . . .	4
3.2.2	Gerarchia dei Processi	5
3.3	Processi vs Thread	6
3.3.1	Implementazione: User vs Kernel Thread	7
3.4	Lo Scheduler e il Context Switch	8
3.4.1	Tipi di Scheduler: Preemptive vs Cooperativo	10
3.4.2	Schema di Funzionamento di un Kernel	10
3.5	Algoritmi di Scheduling	12
3.5.1	First Come, First Served (FCFS) e il problema del Convoy Effect	12
3.5.2	Shortest Job First (SJF) e il problema della non conoscenza preventiva del tempo di esecuzione processi	13
3.5.3	Round-Robin (RR) e il problema della troppa democrazia . .	15
3.5.4	Scheduling a Priorità e il Problema della Starvation	17
3.5.5	Scheduling a Classi di priorità e Multilivello	17
3.6	Scheduling Real-Time	18
3.6.1	Tipologie di Processi Real-Time	18
3.6.2	Esempi di Scheduler Real-Time	19
4	Gestione delle Risorse e Deadlock	20
4.1	Introduzione e Concetto di Risorsa	20
4.1.1	Classificazione delle Risorse	20
4.1.2	Assegnazione e Rilascio	21
4.2	Il Problema del Deadlock	21
4.3	Modellazione: Il Grafo di Holt	22
4.3.1	Grafo di Holt: Notazione Operativa (Grafo Pesato)	23
4.4	Metodi di Gestione dei Deadlock	24
4.4.1	1. Deadlock Detection e Recovery	24
4.4.2	2. Deadlock Prevention (Prevenzione Strutturale)	28
4.4.3	3. Deadlock Avoidance (Elusione): L'Algoritmo del Banchiere	29
4.4.4	4. L'Algoritmo dello Struzzo	35

Capitolo 3

Scheduling, Processi e Thread

3.1 Il Sistema Operativo come Gestore di Risorse

Un sistema operativo agisce fundamentalmente come un gestore delle risorse del calcolatore:

- processore,
- memoria principale e secondaria,
- dispositivi di I/O.

Per governare queste componenti, il S.O. mantiene specifiche strutture dati (tabelle) per tracciarne lo stato:

- **Tabelle di Memoria:** per l'allocazione della RAM ai processi e al kernel, e per i meccanismi di protezione.
- **Tabelle di I/O:** per gestire l'assegnazione dei dispositivi e le code di richieste pendenti.
- **Tabelle del File System:** per tracciare i dispositivi di storage e i file aperti.
- **Tabelle dei Processi:** per monitorare e gestire i programmi in esecuzione. Questa tabella è costituita dall'elenco dei PCB (Process Control Block); pertanto, l'allocazione di risorse specifiche, come i *file descriptor* associati ai file aperti da un processo, viene tracciata e memorizzata proprio all'interno del PCB corrispondente.

3.1.1 Il Descrittore di Processo (PCB)

Per comprendere a fondo il ruolo del PCB, è utile partire da una domanda fondamentale: *qual è la manifestazione fisica di un processo all'interno del calcolatore?* Quando un processo è caricato e attivo, esso è costituito da quattro componenti fisiche ben distinte:

- **Segmento Codice:** la porzione di memoria che contiene le istruzioni da eseguire.
- **Segmenti Dati:** le aree di memoria che contengono le variabili e i dati su cui il programma deve operare.

- **Stack di lavoro:** un'area di memoria dinamica essenziale per la gestione delle chiamate di funzione, il passaggio dei parametri e l'allocazione delle variabili locali.
- **Un insieme di attributi:** tutte le informazioni di controllo necessarie alla gestione del processo, incluse le informazioni (puntatori/limiti) necessarie per descrivere e localizzare in memoria i tre punti precedenti (codice, dati e stack).

Definizione – Process Control Block (PCB)

Questo "insieme di attributi" prende il nome di **Descrittore del Processo** o **PCB (Process Control Block)**. Rappresenta in tutto e per tutto il processo agli occhi del Sistema Operativo.

Le informazioni in esso contenute si dividono in tre aree:

1. **Identificazione:** Il *Process ID (PID)*, un numero o indice univoco. Include anche i PID correlati (es. il PID del processo padre) e l'ID dell'utente proprietario (ovvero colui che ha richiesto l'esecuzione del processo).
2. **Stato del Processore:** Il salvataggio dei registri generali del processore, del Program Counter e dei registri di stato, necessari per riprendere l'esecuzione dopo un'interruzione di un processo.
3. **Controllo:**
 - *Scheduling:* Stato attuale (ready, running, waiting), priorità e puntatori alle code.
 - *Memoria:* Configurazione della MMU e puntatori alle tabelle delle pagine.
 - *Risorse e IPC (interprocess communication):* File aperti, periferiche allocate, stato di semafori e segnali.
 - *Accounting:* Tempo di esecuzione del processo ; oppure il tempo trascorso dall'attivazione del processo.

3.2 Ciclo di Vita di un Processo

Durante la sua esistenza, un processo transita attraverso diversi stati operativi, gestiti dallo scheduler.

- **Running (In esecuzione):** Il processo ha attualmente il controllo della CPU.
- **Ready (Pronto):** Il processo ha tutte le risorse necessarie per eseguire, ma è in attesa che la CPU si liberi.
- **Waiting (In attesa):** Il processo è sospeso in attesa di un evento esterno (es. completamento di un'operazione di I/O) e non può competere per la CPU.

Nota – Code di Processi

I processi non attivi vengono parcheggiati in apposite code. Esiste una **Ready Queue** per i processi pronti all'esecuzione, e diverse **Device Queues** (es. coda del disco, coda del terminale) per i processi in stato *Waiting* che attendono specifiche periferiche.

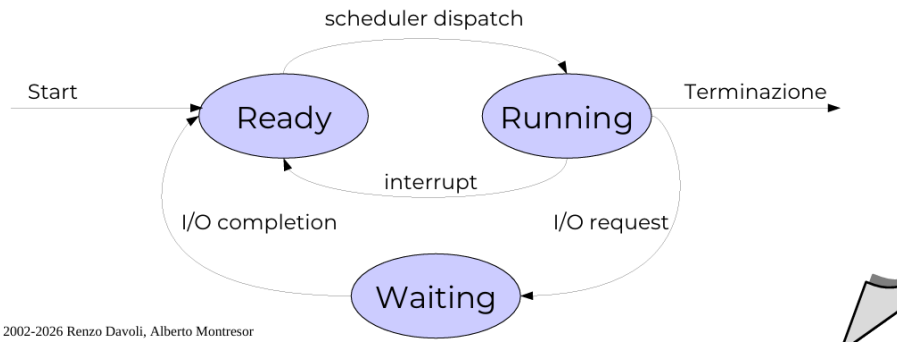


Figura 3.1: **Transizioni di Stato.** Un processo passa da Running a Waiting per una System Call bloccante (I/O), oppure da Running a Ready per la scadenza del tempo o un interrupt. Tornerà Ready al completamento dell'I/O.

3.2.1 Vita di un Processo nello Scheduler: Gestione delle Code

Il percorso di un processo all'interno del sistema operativo può essere visualizzato come un continuo transito tra diverse code gestite dallo scheduler. L'immagine sottostante (3.2) illustra le dinamiche logiche di assegnazione e rilascio della CPU.

Il ciclo vitale ruota attorno a due elementi centrali:

- **Coda Ready:** È la fila in cui risiedono i processi pronti per essere eseguiti. Lo scheduler preleva un processo da questa coda (basandosi sull'algoritmo di scheduling in uso, come FCFS, SJF o Round-Robin [spiegati dopo]) e lo inserisce nella **CPU**.

Una volta che il processo è in esecuzione (stato di *Running* nella CPU), il suo controllo sul processore può interrompersi per tre ragioni principali, evidenziate dai percorsi in uscita nello schema:

1. **System Call Bloccante:** Il processo richiede un'operazione che necessita di tempo (ad esempio, la lettura di un file dal disco o l'attesa di un pacchetto di rete). Non potendo procedere col calcolo, cede volontariamente la CPU e viene inserito in una **Coda di attesa** (spesso associata al dispositivo specifico). Il processo rimane in questa coda finché non si verifica l'**Evento** atteso (es. completamento dell'I/O). Solo a quel punto viene "risvegliato" e rimesso nella **Coda Ready**.
2. **Fine del time slice o altro interrupt:** Nei sistemi *preemptive* (come il Round-Robin), a ogni processo viene assegnato un limite di tempo rigido (time slice o quanto di tempo). Se il processo non termina prima dello scadere di questo tempo, un interrupt del timer hardware lo interrompe forzatamente. Lo stato viene salvato nel PCB e il processo viene rimesso in **Coda Ready**, in attesa di un nuovo turno. Lo stesso percorso si attiva se si verifica un interrupt hardware esterno che richiede l'attenzione immediata della CPU.
3. **System Call Non Bloccante:** Il processo richiede un servizio al Sistema Operativo che può essere soddisfatto immediatamente, senza bisogno di attendere dispositivi lenti. Il sistema esegue rapidamente la routine richiesta e il processo viene reinserito nella **Coda Ready** per poter riprendere l'esecuzione il prima possibile.

Vita di un processo nello scheduler

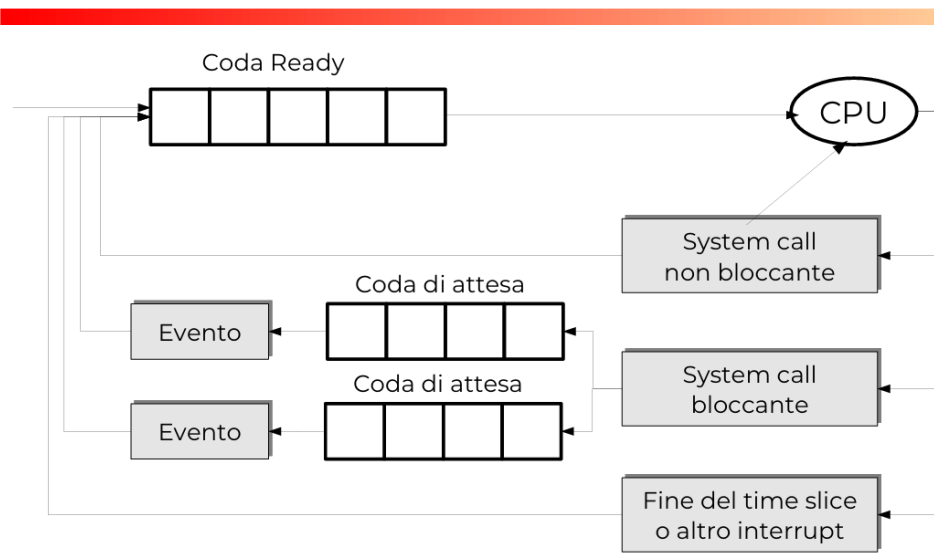


Figura 3.2: **Vita di un processo nello scheduler.** Il diagramma mostra i flussi ciclici tra la Coda Ready, la CPU e le Code di Attesa. I percorsi illustrano come gli eventi di I/O, le chiamate di sistema e i timer determinino lo spostamento del processo tra le varie strutture dati del Kernel.

3.2.2 Gerarchia dei Processi

Nella maggior parte dei sistemi operativi, i processi non esistono come entità isolate, ma sono organizzati secondo una rigorosa struttura gerarchica ad albero. Quando un processo in esecuzione genera un nuovo task, il creatore assume il ruolo di **processo padre**, mentre il nuovo arrivato prende il nome di **processo figlio**.

Questa organizzazione strutturale risponde a una precisa necessità di efficienza: **semplificare il procedimento di creazione dei processi**. Invece di dover costruire un ambiente da zero, allocando nuove risorse e specificando esplicitamente tutti i parametri e le caratteristiche, il sistema sfrutta un meccanismo di derivazione: *il figlio viene creato a immagine e somiglianza del padre*. Ciò significa che tutto ciò che non viene esplicitamente ridefinito durante la creazione, viene automaticamente ereditato dal genitore (ad esempio: variabili d'ambiente, file aperti, directory di lavoro e privilegi utente).

Esempio – L'Albero di UNIX

Un esempio storico e fondamentale di questa architettura è la gerarchia del sistema operativo UNIX. All'avvio della macchina, il kernel crea autonomamente un primissimo processo, storicamente chiamato `init` (PID 1), che funge da radice dell'intero albero.

Da `init` vengono generati a cascata tutti gli altri processi: i servizi in background (*daemon*), i gestori dei terminali (*getty*) e le *shell*. A loro volta, le *shell* generano i processi utente (*user proc*) che possono diramarsi in ulteriori sottoprocessi (*sub process*).

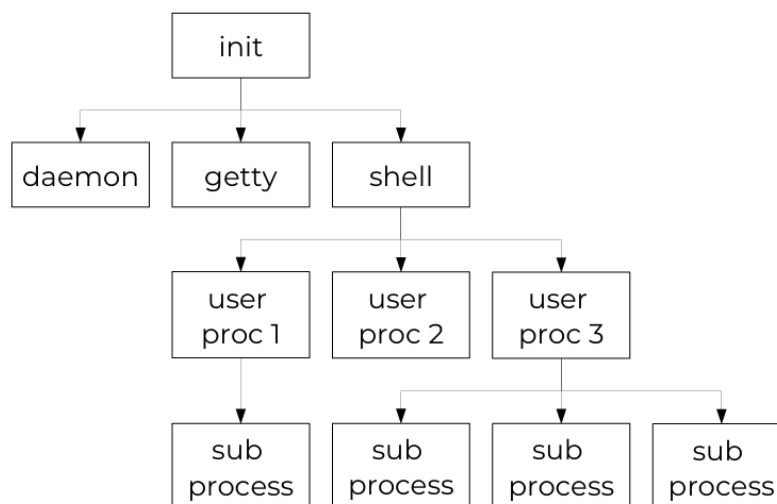


Figura 3.3: **Gerarchia di processi in UNIX.** L'albero mostra visivamente la discendenza genitore-figlio. Ogni nodo (processo) deriva da un padre superiore, risalendo inevitabilmente fino all'unico processo radice, *init*.

3.3 Processi vs Thread

Tradizionalmente un processo ha una singola "linea di controllo" (sequenza di istruzioni). I S.O. moderni permettono processi **Multithreaded**, in cui esistono flussi multipli in parallelo che possono eseguire insiemi di istruzioni differenti.

Definizione – Thread (Lightweight Process)

Un thread è l'unità base di utilizzo della CPU. Sebbene i thread di uno stesso processo **condividano** lo spazio di memoria (codice e dati) e le risorse di I/O, ogni thread possiede privatamente il **proprio Program Counter**, il **proprio stack** e una **propria copia dello stato del processore**.

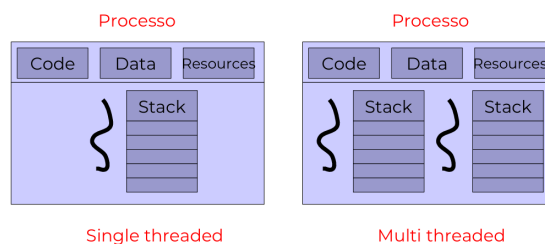


Figura 3.4: Caption

Esempio – Benefici dei Thread

- **Condivisione:** Più thread cooperano facilmente lavorando sulla stessa RAM (es. un web browser usa un thread per la GUI e uno per scaricare i dati).
- **Economia:** Creare thread o effettuare context switch tra thread dello stesso processo è ordini di grandezza più veloce ed economico rispetto

alla gestione di processi interi.

3.3.1 Implementazione: User vs Kernel Thread

- **User Thread:** Gestiti interamente da una libreria in spazio utente. Molto veloci, ma se un thread fa una chiamata di sistema bloccante, il Kernel (ignaro dell'esistenza dei thread) blocca l'intero processo.
- **Kernel Thread:** Il S.O. gestisce direttamente i thread. È più lento (richiede mode switch e quindi di parlare direttamente con il kernel), ma se un thread si blocca, il kernel può schedarne un altro.

Esistono modelli ibridi per mappare i thread utente su quelli kernel: *Many-to-One*, *One-to-One* (può saturare il kernel) e *Many-to-Many* (il più flessibile).

3.4 Lo Scheduler e il Context Switch

Definizione – Scheduling

Lo **Scheduler** è la componente software più importante del kernel che decide l'avvicendamento dei processi (ovvero calcola lo schedule). Noi intendiamo lo scheduler del processore, infatti quando viene richiamato decide quale processo eseguire sulla CPU.

Lo **Schedule** è la sequenza temporale calcolata delle assegnazioni della risorsa (CPU).

lo **Scheduling** è l'azione di calcolare lo schedule

Rappresentazione degli Schedule

Per descrivere visivamente uno schedule , si utilizzano i **diagrammi di Gantt**.

Nel diagramma classico per una singola risorsa (come una singola CPU), l'asse orizzontale rappresenta lo scorrere del tempo. I blocchi contigui indicano quale processo detiene l'uso esclusivo della risorsa in un determinato intervallo temporale.

Nell'esempio illustrato nella figura sottostante, la risorsa viene utilizzata in sequenza:

- Dal processo P_1 a partire dal tempo 0 fino al tempo t_1 .
- Viene quindi riassegnata al processo P_2 , che la occupa dal tempo t_1 fino al tempo t_2 .
- Infine, passa al processo P_3 , dal tempo t_2 fino al tempo t_3 .

Diagramma di Gantt

- per rappresentare uno schedule si usano i diagrammi di Gantt

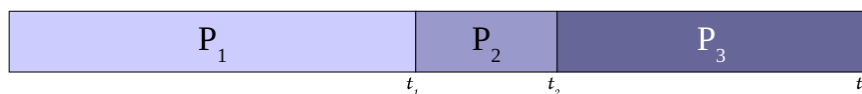


Figura 3.5: **Diagramma di Gantt semplice**. Rappresentazione grafica lineare dell'avvicendamento dei processi (P_1 , P_2 , P_3) nell'utilizzo di una singola CPU nel corso del tempo.

L'intervento dello scheduler è richiesto in casi specifici (System Call, richiesta di I/O, fine I/O o scadenza dell'interval timer).

Quando lo scheduler decide di togliere la CPU al processo A per darla al processo B, il sistema è soggetto a un **Context Switch** (Cambio di Contesto).

- **Mode Switch vs Context Switch:**

- Il **Mode Switch** avviene *tutte le volte* che si verifica un'interruzione (interrupt hardware o software). Il processore passa dalla modalità utente alla modalità supervisore (kernel) per eseguire l'**Interrupt Handler**.
- Il **Context Switch** avviene *solo se*, durante la gestione dell'interrupt, lo scheduler decide di sospendere l'esecuzione del processo corrente per mandare in esecuzione un altro processo pronto.

Esempio – Eventi che causano un Context Switch

Esistono 4 scenari o eventi principali che possono portare all'attivazione dello scheduler e a un potenziale cambio di contesto:

1. Quando un processo passa dallo stato **Running** allo stato **Waiting** (es. per una system call bloccante o una richiesta di I/O).
2. Quando un processo passa dallo stato **Running** allo stato **Ready** (es. a causa di un interrupt o per l'esaurimento del suo time slice).
3. Quando un processo passa dallo stato **Waiting** allo stato **Ready** (es. completamento dell'I/O).
4. Quando un processo **termina** la sua esecuzione.

Nota bene: Nelle condizioni **1 e 4**, il context switch è *obbligatorio* (il processo in esecuzione non può più avanzare, l'unica scelta è selezionare un altro processo). Nelle condizioni **2 e 3**, il context switch è *opzionale* (lo scheduler interviene, ma potrebbe decidere di continuare a eseguire il processo corrente).

- **Operazioni durante il Context switching:** L'operazione ha un costo computazionale per il sistema. Il S.O. deve salvare lo stato completo del processo attuale (registri generali, program counter, ecc.) nel suo **PCB corrispondente**. Successivamente, lo stato del processo selezionato per l'esecuzione viene **caricato dal suo PCB** nei registri fisici del processore.

Context switch

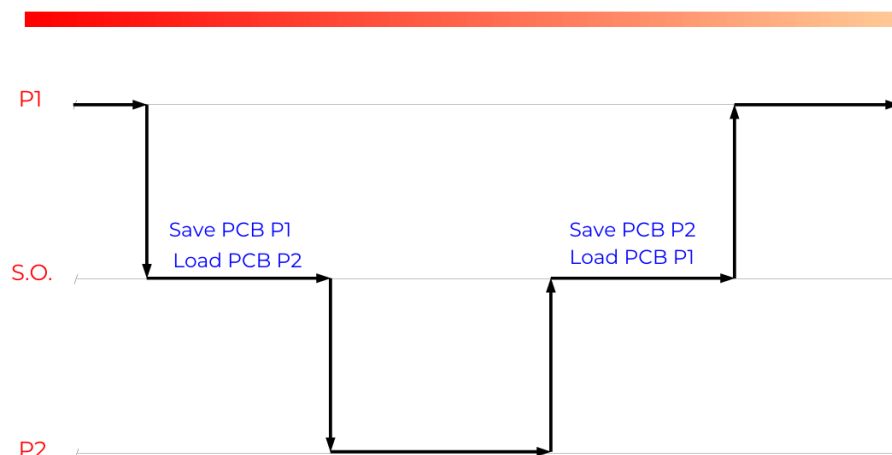


Figura 3.6: **Dinamica del Context Switch.** Il processo P1 (in esecuzione) genera un interrupt o scade il suo timer. Il Sistema Operativo prende il controllo, salva lo stato di P1 nel relativo PCB e carica lo stato di P2 dal suo PCB. Da quel momento, P2 prende il controllo della CPU.

3.4.1 Tipi di Scheduler: Preemptive vs Cooperativo

- **Non-Preemptive (Cooperativo):** Il processo in esecuzione cede il controllo della CPU solo volontariamente (termina o fa una chiamata bloccante, ovvero casi 1 e 4 dell'esempio precedente). Semplice e non richiede timer hardware, ma poco responsivo (es. vecchi Mac OS o Windows 3.1).
- **Preemptive (Con prelazione):** Il S.O. può togliere forzatamente la CPU a un processo in qualsiasi momento (es. alla scadenza di un timer o all'arrivo di un processo più prioritario). È lo standard moderno, ottimizza l'uso delle risorse.

3.4.2 Schema di Funzionamento di un Kernel

Lo schema seguente illustra il ciclo vitale del sistema operativo, evidenziando il continuo passaggio di controllo tra lo *Spazio Kernel* (dove risiedono l'inizializzazione, gli handler e lo scheduler) e lo *Spazio Utente* (dove girano i processi applicativi).

Possiamo suddividere questo funzionamento in sei fasi logiche sequenziali:

1. **Inizializzazione (Initialize):** All'accensione della macchina, il sistema operativo si comporta come un programma sequenziale tradizionale. Esegue le routine di avvio e **crea il primissimo PCB assoluto** (il processo capostipite, come *init*). Terminata questa fase, il codice di inizializzazione esce di scena e non verrà mai più eseguito.
2. **Primo Scheduling:** Il controllo passa per la prima volta allo **Scheduler**. In questa fase embrionale, lo scheduler rileva un solo processo disponibile. Lo seleziona e lo avvia, innescando il passaggio alla modalità utente.
3. **Esecuzione e Interruzione (Processo):** Il processo entra in esecuzione (stato *Running*). Continua a operare finché non si verifica un evento che cede nuovamente il controllo al Kernel. Questi eventi possono essere:
 - Una **System Call** o una richiesta di **I/O** (interruzione volontaria).
 - Un **Interrupt** hardware (es. scadenza dell'interval timer).
4. **Gestione dell'Evento (Interrupt Handler):** Il Kernel riprende il controllo attivando automaticamente l'opportuna routine (*Interrupt Handler*). Se l'interruzione è dovuta a una richiesta di I/O, il processo chiamante viene temporaneamente sospeso e spostato nello stato *Waiting*.
5. **Ritorno allo Scheduler:** Esaurito il compito dell'Interrupt Handler, il flusso di esecuzione converge sempre verso lo Scheduler. Quest'ultimo ispeziona la coda *Ready*. In alcune situazioni, potrebbe trovare la coda vuota (tutti i processi sono bloccati in *Waiting*); in tal caso il sistema attende (es. tramite un *idle process*).
6. **Multitasking e Popolamento Code:** Con l'avanzare dell'esecuzione, i processi attivi andranno a creare nuovi processi figli. Lo Scheduler si troverà così a operare su una coda *Ready* sempre più popolata, potendo alternare l'esecuzione per garantire il multitasking.

Nota – La Natura Anomala del Flusso: Interrupt Handler e Scheduler

Nello spazio Kernel, il flusso di esecuzione non segue le normali regole delle chiamate a funzione (dove a una invocazione corrisponde sempre un **return** al chiamante).

- **Interrupt Handler:** Non viene "chiamato" dal software in modo sequenziale, ma viene attivato asincronamente dall'hardware. Per questo motivo, non ha un **return** tradizionale verso una funzione chiamante. Una volta terminato il suo compito, il flusso viene tipicamente deviato verso lo Scheduler, oppure si conclude con un'istruzione hardware speciale (es. **IRET**) che forza il ripristino dello stato precedente.
- **Scheduler:** Anche questa routine è atipica e **non ha un return**. Non termina restituendo un valore a chi l'ha invocata, ma si conclude "dirottando" fisicamente il flusso di esecuzione: salva lo stato del processore nel PCB del processo appena interrotto, per poi ripristinare e saltare direttamente allo stato (Program Counter) del prossimo processo da mettere in esecuzione.

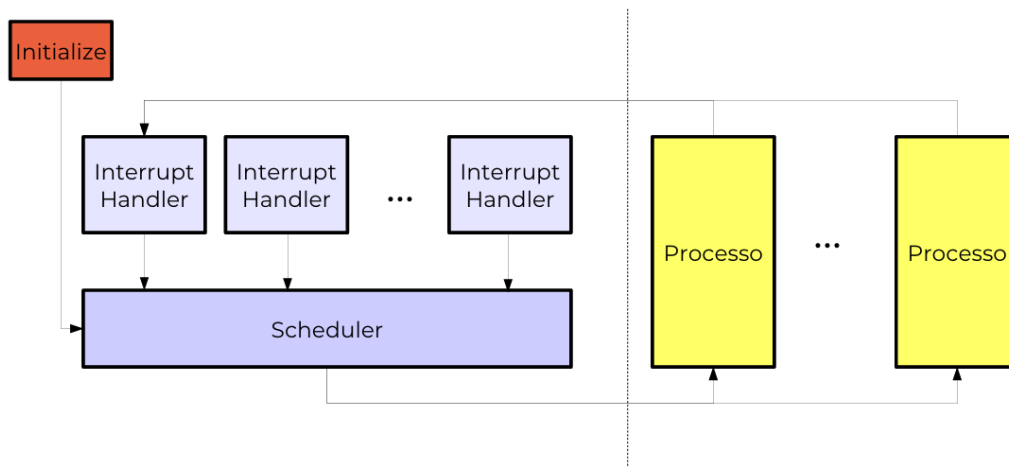


Figura 3.7: **Schema di funzionamento di un kernel.** Si evidenzia il loop infinito tra l'esecuzione dei processi utente e l'intervento delle routine di sistema (Interrupt Handler e Scheduler). A sinistra della linea tratteggiata si trova la parte kernel, a destra quella user

3.5 Algoritmi di Scheduling

Gli algoritmi di scheduling vengono valutati e confrontati in base a specifici criteri prestazionali:

- **Utilizzo della CPU:** Indica la percentuale di tempo in cui il processore è effettivamente occupato a eseguire processi. Questo valore deve essere **massimizzato**.
- **Throughput:** Misura il numero di processi completati per unità di tempo. È un parametro che dipende fortemente dalla lunghezza dei processi stessi e deve essere **massimizzato**.
- **Tempo di Turnaround:** È il tempo totale che intercorre dalla sottomissione di un processo fino alla sua terminazione. Deve essere **minimizzato**.
- **Tempo di Attesa:** Rappresenta la somma dei periodi di tempo trascorsi da un processo all'interno della coda dei processi pronti (*Ready Queue*). Deve essere **minimizzato**.
- **Tempo di Risposta:** È il tempo che intercorre fra la sottomissione di un processo e il momento in cui viene prodotta la prima risposta. È un parametro fondamentale per i sistemi e i programmi interattivi, e deve essere **minimizzato**.

Comportamento dei Processi: CPU e I/O Burst

Durante la sua esecuzione, il comportamento di un processo non è uniforme, ma alterna continuamente due tipologie di fasi:

- **CPU burst:** Periodi in cui il processo utilizza attivamente il processore per eseguire istruzioni e calcoli (es. `load`, `store`, operazioni logico-matematiche). I processi caratterizzati da CPU burst molto lunghi vengono definiti **CPU bound**.
- **I/O burst:** Periodi in cui il processo si blocca in attesa del completamento di operazioni di Input/Output (es. lettura da disco, attesa di pacchetti di rete). I processi caratterizzati da CPU burst molto brevi (e che passano la maggior parte del tempo in attesa di I/O) vengono definiti **I/O bound**.

3.5.1 First Come, First Served (FCFS) e il problema del Convoy Effect

L'algoritmo **FCFS** (Il primo arrivato è il primo ad essere servito) è la politica più semplice da implementare. La CPU viene assegnata ai processi esattamente nell'ordine in cui si presentano, gestendo la *Ready Queue* come una banale coda FIFO (First-In-First-Out).

- **Politica:** Non-preemptive. Una volta che un processo ottiene l'uso della CPU, lo rilascia unicamente quando termina la sua esecuzione o quando si blocca volontariamente per un'operazione di I/O.
- **Svantaggio (Convoy Effect):** Genera tempi medi di attesa e di turnaround generalmente elevati. Se un processo *CPU-bound* molto lungo prende il controllo del processore, tutti i brevi processi *I/O-bound* restano bloccati in coda. Questo crea un "effetto convoglio": le *device queue* si svuotano, i dispositivi hardware rimangono inattivi e l'efficienza globale del sistema crolla.

Esempio – Esempio FCFS

Supponiamo che arrivino in ordine i processi P_1 (burst di 32 ms), P_2 (2 ms) e P_3 (2 ms).

- **Tempo medio di attesa:** $(0 + 32 + 34)/3 = 22$ ms.
- I processi brevissimi sono costretti ad aspettare che il colosso P_1 finisca, alzando drasticamente la media.

Esempio:

- ordine di arrivo: P_1, P_2, P_3
- lunghezza dei CPU-burst in ms: 32, 2, 2
- Tempo medio di turnaround
 $(32+34+36)/3 = 34$ ms
- Tempo medio di attesa:
 $(0+32+34)/3 = 22$ ms

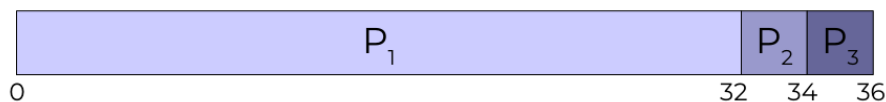


Figura 3.8: **Gantt FCFS e Convoy Effect**. Si nota come l'enorme blocco di calcolo di P_1 ritardi pesantemente i piccoli P_2 e P_3 . Sotto, l'effetto convoglio: le risorse I/O restano inutilizzate in attesa che la CPU si liberi.

3.5.2 Shortest Job First (SJF) e il problema della non conoscenza preventiva del tempo di esecuzione processi

L'algoritmo **SJF** (o *Shortest Next CPU Burst First*) cerca di rimediare al problema del FCFS assegnando la CPU al processo pronto che ha la *minima durata* del prossimo CPU burst.

- **Ottimalità Teorica:** È dimostrato che questo approccio è ottimale, poiché garantisce il minor tempo medio di attesa possibile per un dato insieme di processi.
- **Il Problema Pratico:** È impossibile da implementare alla lettera, poiché il S.O. non può conoscere in anticipo l'esatta durata futura del calcolo di un programma.
- **L'Approssimazione:** Per aggirare il problema, si stima il burst futuro usando una *media esponenziale* basata sulla storia recente dei burst precedenti dello stesso processo (Formula: $T_{n+1} = \alpha t_n + (1 - \alpha)T_n$).

Nota – Approfondimento: Il ruolo di α nella Media Esponenziale

Per stimare la durata del prossimo CPU burst nell'algoritmo SJF, si utilizza la formula:

$$T_{n+1} = \alpha t_n + (1 - \alpha)T_n$$

In questa equazione:

- t_n è la durata **reale** dell'ultimo CPU burst (la *storia recente*).
- T_n è la **previsione** calcolata precedentemente (la *storia passata*).
- α (con $0 \leq \alpha \leq 1$) controlla il **peso relativo** tra passato e presente.

A seconda del valore scelto per α , la strategia del Sistema Operativo cambia radicalmente:

- **Caso $\alpha = 0$ (Solo storia passata):** La formula si riduce a $T_{n+1} = T_n$. Il sistema **ignora completamente** il comportamento attuale del processo. La stima non si aggiorna mai e rimane ancorata al valore iniziale (T_0). È un approccio irrealistico perché non si adatta ai cambiamenti del programma.
- **Caso $\alpha = 1$ (Solo storia recente):** La formula si riduce a $T_{n+1} = t_n$. Il sistema **dimentica tutta la storia passata**. Assume ciecamente che il prossimo burst durerà esattamente quanto l'ultimo appena concluso. Reagisce istantaneamente ai cambiamenti, ma rischia di essere troppo sensibile ai picchi anomali di calcolo.
- **Caso $\alpha = \frac{1}{2}$ (Equilibrio):** La formula diventa $T_{n+1} = \frac{1}{2}t_n + \frac{1}{2}T_n$. Si dà esattamente il 50% di peso a ciò che il processo ha appena fatto e il 50% al suo storico. È il compromesso ideale per filtrare variazioni anomale (grazie alla storia passata) rimanendo comunque reattivi alle nuove fasi del programma (grazie alla storia recente).

- **Problema di Starvation:** Presenta il rischio cronico di inedia (*starvation*). Se continuano ad arrivare numerosi processi molto brevi, un processo lungo rischia di non ottenere mai la CPU.
- **Gestione della Prelazione (Preemption):** L'algoritmo SJF può essere implementato in due varianti distinte:
 - **SJF Non-Preemptive (Classico):** Una volta che la CPU è stata assegnata a un processo, questo ottiene il monopolio temporaneo. Il processo in esecuzione continuerà ininterrottamente fino al completamento del suo CPU burst. Anche se nella coda *Ready* dovesse arrivare un nuovo processo con una durata stimata di calcolo infinitesimale, quest'ultimo sarà costretto ad aspettare.
 - **SJF Preemptive (SRTF - Shortest-Remaining-Time-First):** Il sistema valuta dinamicamente i nuovi arrivi. Se un nuovo processo entra nella coda *Ready* e il suo CPU burst stimato è inferiore al tempo che *rimane* da eseguire al processo attualmente attivo, lo scheduler interviene immediatamente. Il processo corrente subisce un *context switch* forzato (viene sospeso e rimesso in coda) e la CPU viene subito ceduta al nuovo processo più breve.

Esempio

- Tempo medio di turnaround: $(0+2+4+36)/3 = 7$ ms
- Tempo medio di attesa: $(0+2+4)/3 = 2$ ms



Figura 3.9: **Gantt SJF**. A parità di processi dell'esempio precedente, eseguendo prima i task brevi (P_2 e P_3) e lasciando per ultimo il lungo P_1 , il tempo medio d'attesa crolla a soli 2 ms (rispetto ai 22 ms del FCFS).

3.5.3 Round-Robin (RR) e il problema della troppa democrazia

L'algoritmo **Round-Robin (RR)** è il pilastro dei moderni sistemi operativi in *time-sharing* (condivisione del tempo). A differenza del FCFS che favorisce i processi lunghi, il Round-Robin è progettato per garantire **equità** e **reattività**, distribuendo ciclicamente il tempo del processore tra tutti i task presenti nel sistema.

- **Politica Strettamente Preemptive:** A ogni processo pronto viene assegnato un limite massimo di utilizzo ininterrotto della CPU, definito **quanto di tempo** (o *time slice*). L'esecuzione non può mai superare questa soglia senza subire una prelazione (interruzione forzata).
- **Gestione della Coda Ready:** L'insieme dei processi pronti è organizzato come una rigida coda circolare (FIFO). Quando un processo ottiene la CPU, possono verificarsi due scenari:
 1. **Rilascio Volontario:** Il processo ha un CPU burst inferiore al quanto di tempo, oppure effettua una System Call bloccante (es. richiede I/O). In questo caso, lascia spontaneamente la CPU prima dello scadere del timer.
 2. **Rilascio Forzato:** Il processo esaurisce il suo quanto di tempo senza aver completato il suo CPU burst. Il Sistema Operativo interviene, effettua un *context switch* forzato per salvare lo stato del processo, e lo reinserisce **in fondo** alla *Ready Queue*, mettendo in esecuzione il processo successivo.
- **Il Supporto Hardware (Interval Timer):** Per funzionare, il Round-Robin esige che l'hardware fornisca un timer programmabile. Il S.O. imposta questa "sveglia" col valore del time slice non appena avvia un processo. Se il processo non termina da solo, l'interval timer genera un *interrupt hardware* allo scadere del tempo, costringendo il processore a restituire il controllo al kernel (chiamando lo Scheduler).

Nota – Il Dilemma del Tuning: La scelta del Time Slice

La durata del quanto di tempo è il parametro più critico per le prestazioni del sistema:

- **Se è troppo breve:** Il sistema spende una percentuale enorme di tempo e risorse per eseguire continui *context switch* (salvataggio e caricamento dei registri). Questo "overhead" fa crollare l'efficienza globale (la CPU lavora più per il sistema operativo che per l'utente).
- **Se è troppo lungo:** In presenza di molti processi, ogni singolo task subirà lunghi periodi di inattività in attesa del proprio turno. Nei sistemi interattivi, questo causa evidenti "scatti" o ritardi (lag) disastrosi per l'esperienza utente. Di fatto, con un time slice tendente all'infinito, il Round-Robin degenera in un algoritmo FCFS.

E' quindi il più democratico che permette a tutti i processi di proseguire

Esempio – Analisi delle Prestazioni

Immaginiamo di avere tre processi con le seguenti durate dei CPU-burst e un **quanto di tempo impostato a 4 ms**:

- P_1 : burst totale di 10+14 ms
- P_2 : burst totale di 6+4 ms
- P_3 : burst totale di 6 ms

Lo scheduler preleva 4 ms da P_1 , poi passa a P_2 per 4 ms, poi a P_3 per 4 ms, e così via ciclicamente finché i processi non terminano. Il risultato per l'interattività: il **tempo medio di risposta** (il tempo per ricevere la prima reazione dal sistema) crolla a soli **4 ms**, garantendo all'utente l'illusione che i tre programmi stiano avanzando simultaneamente in parallelo.

- tre processi P_1, P_2, P_3
- lunghezza dei CPU-burst in ms (P_1 : 10+14; P_2 : 6+4; P_3 : 6)
- lunghezza del quanto di tempo: 4
- Tempo medio di turnaround $(40+26+20)/3 = 28.66$ ms
- Tempo medio di attesa: $(16+16+14)/3 = 15.33$ ms
 - NB (supponiamo attese di I/O brevi, < 2ms)
- Tempo medio di risposta: 4 ms



Figura 3.10: **Gantt Round-Robin**. Con un time slice di 4 ms, la CPU viene avviata rapidamente. Si nota come P_1 (il processo più lungo) venga frammentato in numerosi blocchi eseguiti a intervalli regolari, permettendo a P_2 e P_3 di non restare bloccati indefinitamente.

3.5.4 Scheduling a Priorità e il Problema della Starvation

Il Round-Robin puro fornisce le stesse possibilità di esecuzione a tutti i processi, ma nella realtà operativa i processi non sono affatto tutti uguali (è **troppo democratico**).

Ad esempio, la visualizzazione di un frame di un video MPEG non dovrebbe mai essere ritardata da un processo in background che sta semplicemente smistando la posta: la lettera può aspettare mezzo secondo senza problemi, il frame video no.

Per gestire queste disparità si introduce lo **Scheduling a Priorità**: a ogni processo viene assegnata una specifica priorità e lo scheduler seleziona sempre il processo pronto col valore di priorità più alto.

Le priorità possono avere due origini:

- **Definite dal Sistema Operativo**: Calcolate internamente basandosi su quantità misurabili. Ad esempio, l'algoritmo SJF è di fatto un sistema a priorità in cui la priorità è determinata dalla brevità stimata del burst.
- **Definite Esternamente**: Imposte direttamente dal livello utente (es. tramite il comando `nice` in Unix) o dalle policy dell'amministratore.

Priorità Statica, Dinamica e Aging

- Una priorità si dice **statica** se non cambia mai durante l'intera vita del processo. Questo approccio rigido presenta un difetto fatale: i processi a **bassa priorità** rischiano la *starvation*, ovvero potrebbero non essere mai eseguiti se nel sistema continuano ad arrivare ininterrottamente processi a priorità maggiore.
- Per prevenire questo blocco cronico si utilizzano le **priorità dinamiche**, che possono variare nel tempo. La tecnica principale è l'**Aging** (invecchiamento): il sistema incrementa gradualmente la priorità dei processi che stazionano da molto tempo nella coda di attesa. Posto che il range di variazione delle priorità sia limitato (es. da 0 a 99), nessun processo rimarrà in attesa per un tempo indefinito, perché prima o poi invecchierà abbastanza da raggiungere la priorità massima e ottenere la CPU. Quando poi il processo che ha raggiunto la priorità più alta torna nella coda ready, tornerà alla priorità più bassa e dovrà riscalarle le priorità.

In caso ci fossero più processi bloccati con la stessa priorità più alta, si sceglie quello che è da più tempo nella coda in attesa.

3.5.5 Scheduling a Classi di priorità e Multilivello

Nei sistemi operativi più strutturati, si creano diverse **classi di processi** con caratteristiche e necessità simili, assegnando a ciascuna classe una specifica priorità. Di conseguenza, la singola coda *Ready* viene scomposta in molteplici "sottocode", una per ogni classe. Uno scheduler a classi di priorità seleziona il processo da eseguire fra quelli pronti della classe a priorità massima che contiene processi

Lo **Scheduling Multilivello** evolve questo concetto: permette di applicare una *politica di scheduling specifica e differente per ogni classe*. Lo scheduler cerca prima la classe a priorità massima che ha almeno un processo pronto; una volta individuata, sceglie il processo da mettere in stato *Running* operando in modo coerente con la politica assegnata a quella specifica coda.

Esempio – Esempio di Code Multilivello

Un sistema potrebbe suddividere i task in quattro classi di priorità (elencate dalla priorità massima alla minima):

1. **Processi Server:** Classe a priorità massima assoluta, gestita con *priorità statica*.
2. **Processi Utente Interattivi:** Classe ad alta priorità, gestita con algoritmo *Round-Robin* per garantire responsività fluida all'input umano.
3. **Altri Processi Utente (Batch):** Lavori lunghi e non interattivi, gestiti con politica *FIFO / FCFS* all'interno della loro coda.
4. **Processo Vuoto (Idle):** L'ultimo livello, gestito con un *FIFO banale*. Gira esclusivamente quando tutte le code superiori sono completamente vuote.

3.6 Scheduling Real-Time

Nei sistemi **Real-Time** (sistemi in tempo reale), il concetto di correttezza è molto più stringente rispetto ai sistemi tradizionali: la validità dell'esecuzione non dipende solamente dall'esattezza logica del risultato calcolato, ma anche dall'**istante temporale** esatto in cui questo risultato viene prodotto ed emesso.

Questa totale dipendenza dal tempo introduce il concetto di **deadline** (scadenza), dividendo le applicazioni in due categorie principali:

- **Hard Real-Time:** Le deadline imposte ai programmi sono assolute e **non devono essere superate in nessun caso**. Il mancato rispetto di una scadenza porta a un fallimento catastrofico dell'intero sistema. Questo vincolo è tipico dei sistemi critici come il controllo di assetto nei velivoli, i software delle centrali nucleari o i macchinari per la cura intensiva dei malati. La conoscenza dei processi in esecuzione è data a priori
- **Soft Real-Time:** Le scadenze sono importanti ma **errori occasionali sono tollerabili** senza causare disastri. Il superamento di una deadline causa un degrado della qualità del servizio, ma il sistema può continuare a funzionare. Ne sono un esempio la ricostruzione e riproduzione di segnali audio/video (streaming multimediale) e le transazioni interattive.

3.6.1 Tipologie di Processi Real-Time

I task all'interno di un sistema in tempo reale si dividono in due grandi famiglie in base alla loro frequenza di attivazione:

- **Processi Periodici:** Sono i processi che vengono riattivati in modo continuo e prevedibile con una cadenza temporale ben precisa, definita **periodo**.

Un esempio classico è la routine di controllo di volo di un aereo, che si risveglia periodicamente per rilevare e rielaborare i parametri dei sensori per prendere decisioni.

- **Processi Aperiodici (o Sporadici):** Non hanno una frequenza fissa e prevedibile, ma vengono scatenati all'improvviso dal verificarsi di un evento esterno (ad esempio, l'attivazione immediata dell'allarme di un rilevatore di pericolo o di fumo). Questi infatti sono caratterizzati da una soglia che una volta superata fa sì che si attivi il processo

3.6.2 Esempi di Scheduler Real-Time

Per garantire il rispetto matematico delle deadline, lo scheduler non può usare algoritmi come il Round-Robin, ma deve utilizzare logiche specializzate. I due algoritmi di riferimento (pensati in primis per i processi periodici) sono:

1. **Rate Monotonic (RM):** È una politica basata sull'assegnazione **statica** delle priorità. La regola di base è lineare: i processi con la frequenza di esecuzione più alta (ovvero con il **periodo più corto**) ricevono la priorità più alta. Ad ogni istante, il sistema esegue il processo pronto con priorità maggiore, effettuando istantaneamente una *preemption* se necessario. Tuttavia, affinché l'algoritmo sia matematicamente valido, richiede condizioni ideali: tutti i processi devono essere reciprocamente indipendenti, l'overhead della preemption deve essere trascurabile, e ogni task deve sempre completare il suo lavoro entro il proprio periodo.
2. **Earliest Deadline First (EDF):** A differenza del Rate Monotonic, l'algoritmo EDF utilizza una politica a **priorità dinamica**. In questo caso, lo scheduler non assegna valori fissi, ma valuta le urgenze istante per istante: viene scelto e messo in esecuzione il processo che ha la **deadline più prossima** a scadere. La priorità relativa tra due o più processi varia dinamicamente in momenti diversi a seconda di quale task sia temporalmente più vicino alla sua scadenza critica.

Capitolo 4

Gestione delle Risorse e Deadlock

4.1 Introduzione e Concetto di Risorsa

Un sistema di elaborazione può essere visto come un vasto bacino di risorse hardware e software che devono essere assegnate in modo efficiente e sicuro ai vari processi in esecuzione. I processi competono costantemente per l'accesso a queste risorse (es. processore, memoria, dischi, interfacce di rete, stampanti, o anche strutture dati interne come i descrittori di processo).

Nota – La Legge del Kansas

Per comprendere l'essenza del problema della condivisione mal gestita, si cita spesso un paradosso normativo (ispirato a una vecchia legge ferroviaria del Kansas): *"Quando due treni si avvicinano l'uno all'altro a un incrocio, entrambi devono fermarsi completamente e nessuno dei due deve ripartire finché l'altro non se ne è andato."* Questa situazione, logicamente irrisolvibile, descrive perfettamente il concetto di stallo o **Deadlock**.

4.1.1 Classificazione delle Risorse

Le risorse non sono tutte identiche e possono essere organizzate e richieste secondo precise logiche:

- **Classi e Istanze:** Le risorse equivalenti vengono raggruppate in *classi* (es. "Stampanti Laser", "Byte di Memoria").

Definizione

Le singole unità fisiche all'interno della classe sono dette *istanze*.
La quantità di istanze definisce la *molteplicità* della classe.

Istanze e molteplicità Un processo, di norma, non può richiedere una specifica istanza fisica, ma solo una risorsa di una specifica classe. Una richiesta per una classe di risorse può essere soddisfatta da qualsiasi istanza libera di quel tipo.

- **Tipi di richieste (Numerosità):**
 - *Richiesta singola:* Si riferisce a una singola risorsa di una classe definita ed è considerato il caso normale.

- *Richiesta multipla*: Si riferisce a una o più classi, e per ogni classe, ad una o più risorse. Questa richiesta deve essere soddisfatta integralmente.
- **Tipi di richieste (Comportamento)**:
 - *Richiesta bloccante*: Il processo richiedente si sospende se non ottiene immediatamente l'assegnazione. La richiesta rimane pendente e viene riconsiderata dalla funzione di gestione ad ogni successivo rilascio.
 - *Richiesta non bloccante*: La mancata assegnazione viene semplicemente notificata al processo richiedente, senza provocare la sua sospensione.
- **Condivisibili vs Non Condivisibili (Seriali)**: Una risorsa seriale non può essere assegnata a più processi contemporaneamente (es. i processori, le sezioni critiche o le stampanti). Una risorsa condivisibile può invece essere usata da più processi simultaneamente (es. un file di sola lettura).

4.1.2 Assegnazione e Rilascio

L'assegnazione delle risorse ai processi può avvenire in due modalità temporali distinte:

- **Assegnazione Statica**: L'assegnazione avviene al momento della creazione del processo e rimane valida fino alla terminazione (es. descrittori di processi o specifiche aree di memoria).
- **Assegnazione Dinamica**: I processi richiedono le risorse durante la loro esistenza, le utilizzano una volta ottenute, e le rilasciano quando non sono più necessarie o al momento della terminazione (es. periferiche I/O o memoria dinamica).

Definizione – Risorse Prerilasciabili (Preemptable)

Una risorsa si dice **prerilasciabile** se la funzione di gestione può sottrarla ad un processo prima che questo l'abbia effettivamente rilasciata. Il processo che subisce il prerilascio deve sospendersi e la risorsa gli sarà successivamente restituita.

Una risorsa è prerilasciabile se il suo stato non si modifica durante l'utilizzo oppure se il suo stato può essere facilmente salvato e ripristinato (es. il processore o i blocchi di memoria). Le risorse **non prerilasciabili** (es. le stampanti o le classi di sezioni critiche) non possono essere sottratte al processo al quale sono assegnate, poiché il loro stato non può essere salvato e ripristinato in modo sicuro.

4.2 Il Problema del Deadlock

Il **Deadlock** (stallo) è una situazione anomala in cui un gruppo di processi è bloccato indefinitamente perché ognuno attende una risorsa che è attualmente trattenuta da un altro processo del gruppo.

Affinché un deadlock possa verificarsi, devono essere soddisfatte **contemporaneamente quattro condizioni necessarie**:

1. **Mutua esclusione / non condivisibili:** Le risorse coinvolte non sono condivisibili (sono di tipo seriale).
2. **Assenza di prerilascio:** Le risorse non possono essere sottratte forzatamente; devono essere rilasciate volontariamente dai processi.
3. **Richiesta bloccante (Hold and Wait):** Un processo che possiede già delle risorse può effettuare nuove richieste e bloccarsi in attesa di ottenerle, senza rilasciare quelle che già detiene.
4. **Attesa circolare:** Esiste un ciclo chiuso di processi $P_0, P_1, \dots, P_n$ tale per cui P_0 attende una risorsa posseduta da P_1 , P_1 ne attende una da P_2 , e così via, fino a P_n che attende una risorsa posseduta da P_0 .

Nota – Condizioni Necessarie e Sufficienti per il Deadlock

L'insieme di queste quattro condizioni è **necessario e sufficiente**. Questo significa che le condizioni devono valere tutte contemporaneamente affinché un deadlock si presenti nel sistema. Se anche solo una di esse viene a mancare, lo stallo strutturale non può verificarsi.

4.3 Modellazione: Il Grafo di Holt

Per rilevare e studiare i deadlock, si utilizza uno strumento matematico chiamato **Grafo di Holt** (o grafo di assegnazione delle risorse). È un grafo:

- **diretto**, ovvero che gli archi hanno una sola direzione
- **bipartito** (i nodi sono divisi in due insiemi: processi e classi di risorse).

Le componenti che costituiscono tale grafo sono:

- I **Processi** sono rappresentati da cerchi.
- Le **Classi di Risorse** sono rappresentate da rettangoli.
- Le **Istanze** di una risorsa sono rappresentate da punti (pallini neri) all'interno del rettangolo della classe.
- **Archì di Assegnazione:** Un arco che parte da un punto (istanza/risorsa) e va verso un cerchio (processo) indica che la risorsa è *assegnata* a quel processo.
- **Archì di Richiesta:** Un arco che parte da un cerchio (processo) e punta a un rettangolo (classe e non ad una singola risorsa) indica che il processo *ha richiesto* una risorsa di quel tipo ed è bloccato in attesa.

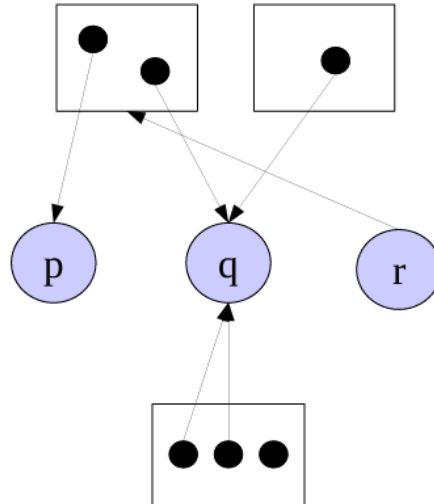


Figura 4.1: **Esempio di Grafo di Holt Generale.** I processi (cerchi) richiedono risorse alle classi (rettangoli), mentre specifiche istanze (puntini neri) sono già assegnate ad altri processi.

4.3.1 Grafo di Holt: Notazione Operativa (Grafo Pesato)

Nell'implementazione reale all'interno del Sistema Operativo, il grafo di Holt non viene memorizzato tenendo traccia visiva di ogni singolo puntino, ma viene ottimizzato e gestito come un **grafo pesato**. Questo approccio associa dei valori numerici (pesi) sia ai nodi delle risorse sia agli archi direzionali.

Le regole di lettura di questa notazione operativa sono le seguenti:

- **Archi Processo $\rightarrow$ Classe (Richiesta):** Il peso riportato sull'arco indica la *molteplicità della richiesta*, ovvero il numero esatto di istanze di quella specifica classe che il processo sta attendendo.
- **Archi Classe $\rightarrow$ Processo (Assegnazione):** Il peso riportato sull'arco indica la *molteplicità dell'assegnazione*, ovvero quante istanze fisiche di quella classe sono attualmente in possesso di quel processo.
- **Nodi Classe di Risorse (Rettangoli):** Il numero scritto all'interno del rettangolo della classe **non** rappresenta il totale delle istanze esistenti, ma indica esclusivamente il numero di *risorse non ancora assegnate* (ovvero le istanze attualmente libere e pronte all'uso).

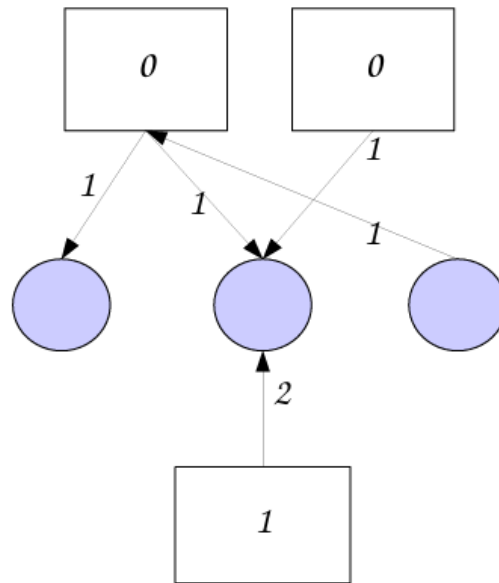


Figura 4.2: **Notazione Operativa del Grafo di Holt.** Esempio di grafo pesato in cui i rettangoli mostrano le risorse residue (0, 0, 1) e gli archi mostrano le quantità di risorse richieste (peso 1 o 2) o già assegnate (peso 1).

4.4 Metodi di Gestione dei Deadlock

Per affrontare il problema dei deadlock all'interno dei sistemi operativi, esistono tre macro-approcci o filosofie principali:

1. **Detection and Recovery:** permettere al sistema di entrare in stallo, usare un algoritmo per rilevarlo e poi eseguire un'azione di ripristino.
2. **Prevention / Avoidance:** impedire a priori (strutturalmente) o dinamicamente che il sistema possa mai entrare in uno stato di deadlock.
3. **Algoritmo dello Struzzo:** ignorare del tutto il problema.

4.4.1 1. Deadlock Detection e Recovery

In questo approccio, il S.O. non fa nulla per impedire i deadlock, ma si limita a monitorare le richieste tramite il Grafo di Holt per accorgersi se uno stallo si è verificato.

1) Fase di Rilevamento (Detection)

La presenza di un ciclo nel grafo di Holt è un indicatore chiave, ma la sua interpretazione dipende dal numero di istanze per classe:

- **Una sola risorsa per classe:** se le risorse sono a richiesta bloccante, non condivisibili, non prerilasciabili allora il sistema è in deadlock **se e solo se** il grafo contiene un ciclo (attesa circolare). In questo caso il Grafo di Holt può essere semplificato in un *Grafo Wait-For* (eliminando i nodi risorsa e collegando direttamente i processi tra loro).
- **Più risorse per classe:** La presenza di un ciclo è una condizione necessaria ma **non sufficiente** (il ciclo potrebbe rompersi se un processo esterno libera un'istanza).

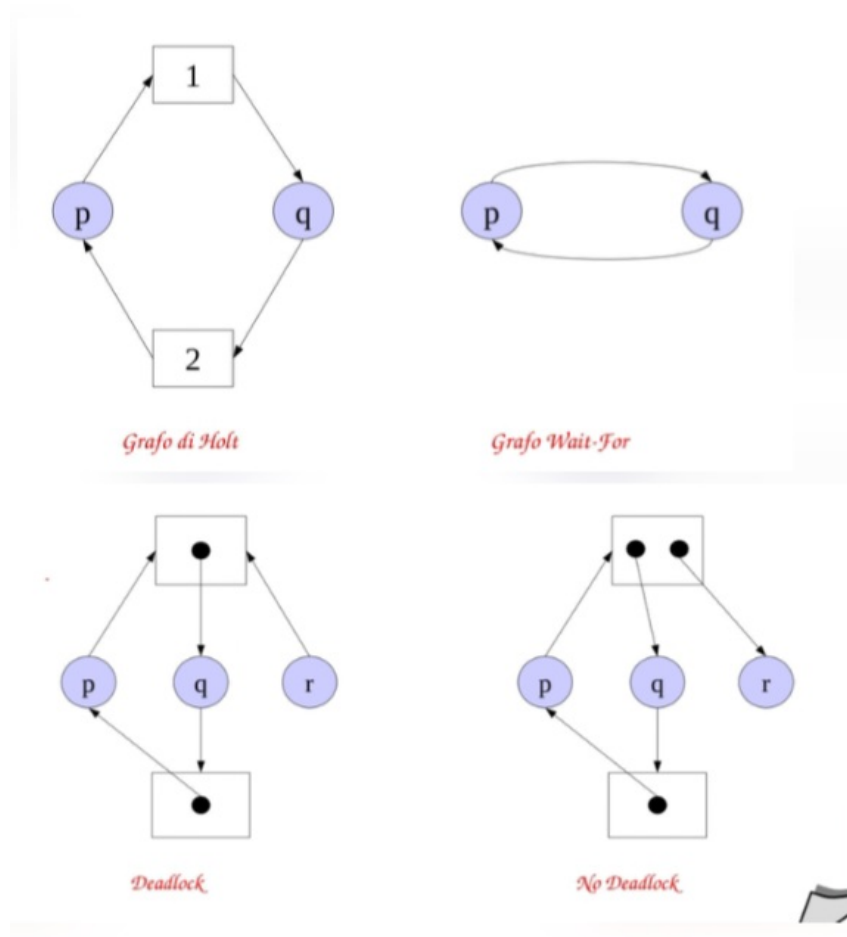


Figura 4.3: **Dal Grafo di Holt al Grafo Wait-For.** Rimuovendo i nodi risorsa e unendo gli archi, il ciclo di richieste tra i processi risulta immediatamente visibile.

Per avere la certezza matematica del deadlock con risorse multiple, si applica la **Riduzione del Grafo**.

Definizione – Riducibilità di un Grafo di Holt

Un grafo di Holt si dice **riducibile** se esiste almeno un nodo processo con solo **archi entranti**.

La **riduzione** consiste nell'eliminare tutti gli archi di tale nodo e riassegnare le risorse ad altri processi che richiedevano le risorse che erano "detenute" (poiché, logicamente, un processo che utilizza una risorsa prima o poi la rilascerà al termine dell'esecuzione).

Nota – Teorema della Riducibilità:

Se le risorse sono a richiesta bloccante, non condivisibili e non prerilasciabili, lo stato **non è di deadlock se e solo se** il grafo di Holt è **completamente riducibile** (ovvero esiste una sequenza di passi di riduzione che elimina tutti gli archi del grafo).

Se il grafo non è completamente riducibile, i nodi rimanenti confermano lo stallo. In particolare, se il sistema ha *una sola richiesta sospesa per processo*, la presenza del deadlock è definita dalla presenza di un **Knot**.

Definizione – Knot (Nodo Gordiano)

Un **knot** è un sottoinsieme (non banale) di nodi M tale che, per ogni nodo n in M , l'insieme di raggiungibilità $R(n)$ [ovvero l'insieme di nodi raggiungibili da n] è uguale a M stesso. In altre parole: partendo da un qualunque nodo di M , si possono raggiungere **tutti e soli i nodi di M** , senza alcuna via d'uscita verso nodi esterni.

Teorema del Knot: Dato un grafo di Holt con una sola richiesta sospesa per processo, se le risorse sono a richiesta bloccante, non condivisibili e non prerilasciabili, allora il grafo rappresenta uno stato di deadlock **se e solo se** esiste un knot.

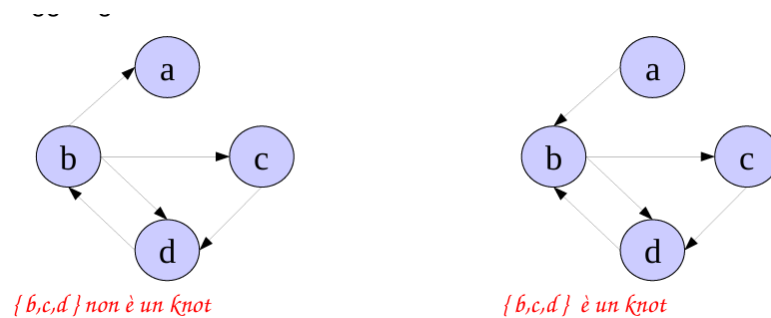


Figura 4.4

Esempio – Esempio di Riduzione del Grafo

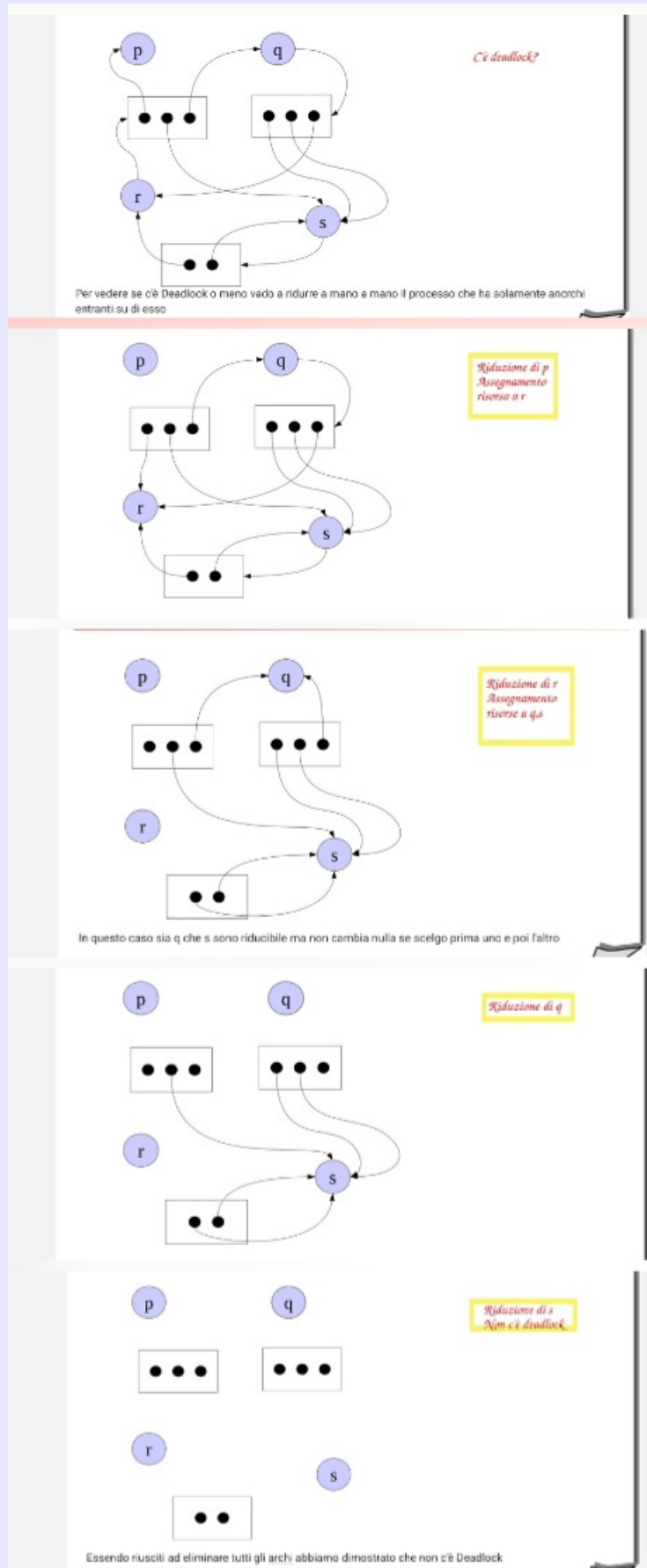


Figura 4.5: **Riduzione del Grafo.** Simulazione della terminazione di un processo e liberazione delle sue risorse.

2) Fase di Ripristino (Recovery)

Una volta rilevato, il deadlock deve essere risolto. Il ripristino può essere manuale (l'operatore esegue azioni correttive) o automatico. I meccanismi automatici includono:

- **Terminazione dei processi:** Può essere *totale* (uccisione di tutti i task coinvolti) o *parziale* (eliminazione di un processo alla volta finché il deadlock non scompare). È un'operazione costosa perché si perde il lavoro fatto dai processi in esecuzione da molto tempo e può lasciare le risorse in uno stato incoerente (es. se terminati nel mezzo di una sezione critica).
- **Checkpoint/Rollback:** Lo stato dei processi viene periodicamente salvato su disco (checkpoint). In caso di stallo, il sistema ripristina (rollback) uno o più processi ad uno stato precedente, fino a quando il deadlock non scompare.

4.4.2 2. Deadlock Prevention (Prevenzione Strutturale)

Questo approccio consiste nell'impedire radicalmente al sistema di entrare in uno stato di deadlock, eliminando a priori (strutturalmente) almeno una delle quattro condizioni necessarie.

- **Attaccare la Mutua Esclusione:** Si cerca di permettere la condivisione delle risorse. Un classico esempio è lo *spool di stampa*: tutti i processi "pensano" di usare contemporaneamente la stampante scrivendo su un buffer, eliminando l'attesa.
Problemi: Lo spooling non è sempre applicabile (ad esempio, non si può usare per i descrittori di processi) e spesso sposta semplicemente il problema del deadlock verso un'altra risorsa (es. l'esaurimento dello spazio fisico sul disco dedicato allo spooling).
- **Attaccare la Richiesta Bloccante (Hold and Wait):** Si impone una politica di **Allocazione Totale**. Un processo è obbligato a richiedere tutte le risorse di cui avrà bisogno fin dall'inizio della sua computazione.
Problemi: Non sempre l'insieme di richieste è noto a priori. Inoltre, questa tecnica è altamente inefficiente e riduce drasticamente il parallelismo: anche se un processo ha necessità di una risorsa per pochissimo tempo, è costretto ad allocarla e bloccarla per tutta la propria esistenza.
- **Attaccare l'Assenza di Prerilascio:** Si permette al sistema operativo di sottrarre forzatamente la risorsa al processo che la detiene.
Problemi: Come visto in precedenza, non sempre è fisicamente o logicamente possibile (es. interrompere una stampante a metà) e può richiedere delicati interventi manuali.
- **Attaccare l'Attesa Circolare:** Si utilizza un'architettura ad **Allocazione Gerarchica**. Alle classi di risorse vengono associati dei valori di priorità fissi. La regola d'oro è che ogni processo può allocare *solamente* risorse di priorità superiore a quelle che già possiede. **Se un processo vuole richiedere una risorsa a priorità inferiore, deve prima rilasciare tutte le risorse con priorità uguale o superiore a quella desiderata.**

Problemi: È una soluzione altamente inefficiente, poiché l'indisponibilità di una risorsa ad alta priorità ritarda inutilmente processi che detengono già altre risorse ad alta priorità.

4.4.3 3. Deadlock Avoidance (Elusione): L'Algoritmo del Banchiere

A differenza della *Prevention* (che agisce *a priori* sulle regole strutturali), l'**Avoidance** interviene **dinamicamente** durante l'esecuzione: prima di assegnare una risorsa a un processo, il sistema verifica se l'operazione può creare un pericolo di deadlock. Se rileva un rischio, la richiesta viene ritardata finché il sistema non si trova di nuovo in uno stato sicuro.

Il modello teorico di riferimento per l'Avoidance è l'**Algoritmo del Banchiere**, ideato da E. W. Dijkstra nel 1965.

La Metafora del Banchiere

Immaginiamo un banchiere che gestisce un **capitale fisso** (le risorse del sistema) concedendo crediti a un gruppo **prefissato** di clienti (i processi).

- Ogni cliente dichiara in anticipo la sua **necessità massima** di denaro (limite di credito), che non può mai superare il capitale totale della banca.
- I clienti effettuano transazioni nel tempo, alternando richieste di prestito a restituzioni; il denaro prestato non eccede mai il massimo dichiarato.
- Una volta ottenuto il credito, ogni cliente **garantisce la restituzione in un tempo finito**.
- Il banchiere concede un prestito *solo se* mantiene la garanzia matematica di poter soddisfare le richieste massime di **tutti** i clienti in tempo finito: questo è il principio fondante dell'algoritmo.

Modello Matematico (caso a singola classe di risorse)

Formalizziamo il problema per un sistema con un'unica classe di risorse. Definiamo le seguenti variabili:

- N : numero totale dei clienti (processi).
- IC : capitale iniziale totale (*Initial Capital*).
- C_i : limite massimo di credito del cliente i (con $C_i \leq IC$).
- p_i : prestito attualmente erogato al cliente i (con $p_i \leq C_i$).
- $n_i = C_i - p_i$: credito residuo, ovvero il massimo che il cliente i potrebbe ancora richiedere prima di raggiungere il suo tetto.
- $COH = IC - \sum_{i=1}^N p_i$: *Cash On Hand*, il saldo di cassa attualmente disponibile (può diventare 0, ma non negativo).

Definizione – Stato SAFE (Sicuro)

Uno stato si definisce **SAFE** se esiste almeno una permutazione $s(1), s(2), \dots, s(N)$ dei processi tale per cui le risorse disponibili, sommate a quelle progressivamente restituite dai processi man mano che terminano, siano sempre sufficienti a coprire le richieste residue n_i di ciascun processo

nella sequenza.

Matematicamente, si calcola il vettore `avail` passo per passo:

- `avail[1] = COH` (disponibilità iniziale).
- `avail[j + 1] = avail[j] + ps(j)` per $j = 1, \dots, N - 1$ (la disponibilità cresce man mano che ogni processo termina e restituisce il suo prestito).

La condizione di sicurezza è soddisfatta se, per ogni passo j :

$$n_{s(j)} \leq \text{avail}[j] \quad \forall j = 1, \dots, N$$

Se la simulazione di un'assegnazione porta a uno stato SAFE, il S.O. concede immediatamente la risorsa.

Definizione – Stato UNSAFE (Insicuro)

Se *non* è possibile trovare alcuna permutazione s che soddisfi $n_{s(j)} \leq \text{avail}[j]$ per tutti i processi, lo stato è **UNSAFE**.

Attenzione: uno stato UNSAFE è *condizione necessaria ma non sufficiente* per il deadlock. Il sistema potrebbe comunque evolvere senza stalli se i processi restituissero le risorse prima di raggiungere il loro massimo dichiarato C_i . Tuttavia, il S.O. non può avere la certezza matematica di ciò, quindi per sicurezza nega l'assegnazione e mette la richiesta in attesa.

Nota – Regola pratica per verificare lo stato SAFE (singola valuta)

Per trovare una sequenza sicura nel caso a singola classe di risorse, è sufficiente **ordinare i processi per valori crescenti di n_i** . Si può dimostrare che se esiste una qualsiasi sequenza che porta allo stato SAFE, allora anche quella ordinata per n_i crescente la trova. Questo riduce la verifica a un singolo ordinamento, senza dover esplorare tutte le $N!$ permutazioni.

Esempio – 1 – Situazione Iniziale

Il seguente scenario rappresenta il punto di partenza del sistema: nessun cliente ha ancora ricevuto nulla.

Parametro	Simbolo	Valore
Capitale iniziale	IC	150
Saldo di cassa	COH	150

Cliente i	Limite max C_i	Prestito attuale p_i	Credito residuo n_i
1	100	0	100
2	20	0	20
3	30	0	30
4	50	0	50
5	70	0	70

In questo stato iniziale il sistema è ovviamente SAFE: il $COH = 150$ è

sufficiente a coprire qualsiasi richiesta da parte di chiunque.

– **Stato SAFE con COH = 0**

Dopo una serie di assegnazioni, il sistema si trova nella seguente configurazione con il saldo di cassa completamente esaurito ($COH = 0$):

Parametro	Simbolo	Valore
Capitale iniziale	IC	150
Saldo di cassa	COH	0

Cliente i	C_i	p_i	n_i
1	100	70	30
2	20	10	10
3	30	30	0
4	50	10	40
5	70	30	40

Verifica: $\sum p_i = 70 + 10 + 30 + 10 + 30 = 150 = IC$, quindi $COH = 0$. Pur avendo il saldo azzerato, il sistema è ancora **SAFE**. Ordinando per n_i crescente, ovvero credito residuo, si ottiene la sequenza $s = \langle 3, 2, 1, 4, 5 \rangle$ (o equivalentemente $\langle 3, 2, 1, 5, 4 \rangle$, essendo $n_4 = n_5$).

Passo j	Processo $s(j)$	$n_{s(j)}$	$avail[j]$	$n_{s(j)} \leq avail[j]$?
1	3	0	0	✓
2	2	10	$0 + p_3 = 0 + 30 = 30$	✓
3	1	30	$30 + p_2 = 30 + 10 = 40$	✓
4	4	40	$40 + p_1 = 40 + 70 = 110$	✓
5	5	40	$110 + p_4 = 110 + 10 = 120$	✓

Interpretazione: alla sequenza parte con $avail[1] = COH = 0$. Il cliente 3 ha già tutto il credito massimo ($n_3 = 0$), quindi può terminare subito e restituire i suoi $p_3 = 30$ al banchiere. Questo libera 30 unità che rendono possibile soddisfare il cliente 2 ($n_2 = 10 \leq 30$), e così via a cascata. Alla fine, tutti i clienti vengono soddisfatti in tempo finito: **lo stato è SAFE**.

Esempio – 2 – Stato UNSAFE ($COH = 10$)

Modifichiamo leggermente la distribuzione dei prestiti, portando il COH a soli 10 unità:

Parametro	Simbolo	Valore
Capitale iniziale	IC	150
Saldo di cassa	COH	10

Cliente i	C_i	p_i	n_i
1	100	65	35
2	20	10	10
3	30	5	25
4	50	15	35
5	70	35	35

Verifica: $\sum p_i = 65 + 10 + 5 + 15 + 35 = 130$, quindi $COH = 150 - 130 = 20$. Ordinando per n_i crescente, la sequenza candidata è $s = \langle 2, 3, 1, 4, 5 \rangle$ (o equivalentemente con 1,4,5 in qualsiasi ordine tra loro poiché $n_1 = n_4 = n_5 = 35$):

Passo j	Processo $s(j)$	$n_{s(j)}$	$avail[j]$	$n_{s(j)} \leq avail[j]$?
1	2	10	10	✓
2	3	25	$10 + p_2 = 10 + 10 = 20$	✓
3	1	35	$20 + p_3 = 20 + 5 = 25$	
4	4	35	—	
5	5	35	—	

Al passo 3, il credito residuo del cliente 1 è $n_1 = 35$, ma la disponibilità è solo $avail[3] = 25$: la condizione $n_{s(3)} \leq avail[3]$ **fallisce**. Non esiste alcuna altra permutazione valida (lo si può verificare per esclusione, poiché al passo 3 nessun processo rimanente ha $n_i \leq 25$). Lo stato è quindi **UNSAFE**: il S.O. avrebbe negato l'assegnazione che ha portato a questo stato.

Esempio – 3 – UNSAFE non implica Deadlock

Questo esempio illustra il fatto cruciale che uno stato UNSAFE *non* è necessariamente uno stato di deadlock: il deadlock si verificherebbe solo se i processi usassero *effettivamente* tutto il credito dichiarato.

Parametro	Simbolo	Valore
Capitale iniziale	IC	150
Saldo di cassa	COH	0

Cliente i	C_i	p_i	n_i
1	100	75	25
2	20	10	10
3	30	5	25
4	50	15	35
5	70	45	25

Con $COH = 0$ e tutti i $n_i \geq 10$, nessun processo può essere soddisfatto con la disponibilità attuale: si tratta di uno stato UNSAFE.

Tuttavia, supponiamo che il cliente 5 — al di là di quanto dichiarato come massimo — decida spontaneamente di restituire *prima del previsto* l'intero prestito ($p_5 = 45$). Il COH torna improvvisamente a 45, e la sequenza $\langle 2, 1, 3, 5, 4 \rangle$ diventa praticabile: il sistema rientra nello stato SAFE senza che si sia mai verificato un deadlock.

Questo dimostra che lo stato UNSAFE è *condizione necessaria ma non sufficiente* per il deadlock: il S.O. lo evita per prudenza matematica, ma il sistema avrebbe potuto comunque procedere.

Estensione: Algoritmo del Banchiere Multivaluta

L'algoritmo si estende naturalmente al caso di **più classi di risorse** (il banchiere gestisce più valute: euro, dollari, yen, ...; nel sistema operativo sono le diverse tipologie di risorsa). Tutte le variabili diventano **vettori**, un elemento per ciascuna classe di risorsa k :

- IC : vettore del capitale iniziale per ogni classe.
- C_i, p_i, n_i : vettori per ogni cliente.
- $COH = IC - \sum_{i=1}^N p_i$: vettore del saldo di cassa.

La definizione di stato SAFE rimane la stessa (deve esistere una permutazione s tale che $n_{s(j)} \leq \text{avail}[j]$ componente per componente), ma la **regola pratica dell'ordinamento per n_i crescente non è più applicabile**, perché l'ordine potrebbe essere diverso per ciascuna valuta.

Nota – U

n cliente può anche non restituire tutte le valute assieme, a patto che una volta restituita una valuta, questa non può essere richiesta nuovamente fino a quando non ha finito di restituire anche le altre

Definizione – Algoritmo greedy per il caso multivaluta

Al passo j , si sceglie *un qualsiasi* processo non ancora servito per cui $\mathbf{n}_{s(j)} \leq \mathbf{avail}[j]$ (confronto componente per componente). Se esiste almeno un tale processo, lo si aggiunge alla sequenza, si aggiorna $\mathbf{avail}[j+1] = \mathbf{avail}[j] + \mathbf{p}_{s(j)}$, e si prosegue. Se ad un certo passo *nessun* processo rimanente è soddisfacibile, lo stato non è SAFE.

Teorema: se durante la costruzione della sequenza si giunge a un punto in cui nessun processo è soddisfacibile, allora lo stato non è SAFE, ovvero non esiste alcuna sequenza alternativa che possa completarla.

(Dimostrazione per assurdo: se esistesse una sequenza SAFE C' , il primo processo h di C' non ancora inserito in C sarebbe soddisfacibile con le risorse disponibili alla fine di C , contraddicendo l'ipotesi che C non sia ulteriormente estendibile.)

Siano C e C' le sequenze di processi come in figura

Sia $H = \{p \in \text{processi} \mid p \notin C\}$

sia h il primo elemento di H che compare in C'

tutti gli elementi di C' prima di h compaiono in C . Chiamiamo C'' il segmento iniziale di C' fino al punto precedente l'applicazione di h

le risorse disponibili all'applicazione di h in C' sono $\mathbf{avail}(C'') = \mathbf{COH} + \sum_{p \in C''} \mathbf{p}_i$; h è applicabile quindi $\mathbf{n} \leq \mathbf{avail}(C'')$

ma $\mathbf{avail}(C) = \mathbf{COH} + \sum_{p \in C} \mathbf{p}_i$, quindi se $\mathbf{avail}(C) \leq \mathbf{avail}(C'')$ allora h è applicabile alla fine di C contro l'ipotesi che C non fosse ulteriormente estendibile

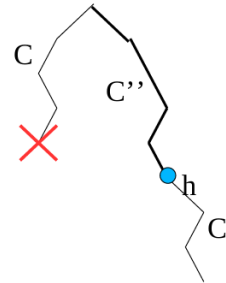


Figura 4.6: Dimostrazione algoritmo del banchiere

4.4.4 4. L'Algoritmo dello Struzzo

Nonostante l'eleganza teorica dell'Algoritmo del Banchiere e dei metodi di *Prevention*, nella realtà industriale queste soluzioni sono considerate **impraticabili** (in quanto evitare deadlock può avere costo troppo elevato) per i sistemi operativi *general-purpose*.

- **Il Problema Ingegneristico:** implementare tecniche di Avoidance ha un costo computazionale elevato e introduce grande rigidità. Costringere ogni processo a dichiarare in anticipo il suo picco massimo di utilizzo delle risorse è irrealistico nella pratica, e i benefici ottenuti non giustificano la penalizzazione continua delle performance.
- **La Soluzione Pratica adottata da alcuni:** si adotta il cosiddetto **Algoritmo dello Struzzo**, che consiste letteralmente nel “nascondere la testa sotto la sabbia”: il S.O. fa finta che il problema dei deadlock non possa mai verificarsi.

Esempio – Sistemi Reali

I deadlock sono eventi statisticamente rari su una tipica macchina desktop. Per questo motivo, i progettisti dei sistemi **UNIX**, **Windows**, **Linux** e persino macchine virtuali come la **JVM** (Java Virtual Machine) optano per l'Algoritmo dello Struzzo. Il sistema operativo scarica sull'utente l'onere di risolvere il blocco, ad esempio aprendo un Task Manager e forzando la terminazione dell'applicazione bloccata.

Approccio	Idea	Costo / Problemi
Detection & Recovery	Lascia entrare il deadlock, poi lo rileva e ripristina	Overhead continuo; rollback può corrompere lo stato
Prevention	Elimina strutturalmente una delle 4 condizioni	Inefficiente; rigido; spesso inapplicabile
Avoidance (Banchiere)	Verifica dinamicamente la safety prima di ogni assegnazione	Costo $O(N^2)$ per richiesta; richiede la dichiarazione del max
Struzzo	Ignora il problema	Nessuno (i deadlock sono rari in pratica)